Frontend Development

EduGenie - AI Learning Assistant

1. Objective

The frontend is the part of EduGenie that students actually see. It is a single, server-rendered page built with HTML, CSS and Jinja2. The goal was a clean interface where a learner can pick a tool, type a query and read the result without any training.

2. Technologies Used

Technology	File	Role
HTML5	templates/index.html	Page structure, form and answer area
CSS3	static/style.css	Layout, colours, spacing, hover effect
Jinja2	templates/index.html	Fills in the response, prompt, task and model badge; keeps the selected option
FastAPI static mount	Main.py	Serves /static/style.css to the browser
Jinja2Templates	Main.py	Renders index.html for GET / and POST /generate

3. Page Structure

index.html is a standard HTML5 page with a UTF-8 character set and a responsive viewport meta tag. Everything is placed inside one div with the class **container**:

Element	Content in the code	Purpose
Heading (h1)	"EduGenie" with a genie icon	Brand name
Subtitle (p.subtitle)	"Multi-Model AI Learning Portal"	Tells the user the portal uses more than one model
Task selector (select#task)	Five options: Concept Explanation (Local LaMini-Flan-T5), General Q&A, Summarization, Quiz Generation, Learning Path Generation (each Gemini 3.8 flash)	Chooses the tool and shows which model will answer
Query box (textarea#prompt)	Placeholder "Type a concept, question, or topic...", required	Input for the question, topic or text
Submit button	"Submit Request"	Posts the form to /generate
Answer box (div.response-box)	Model badge, "Output:" label and the response	Shows the result; displayed only when a response exists

4. How the Template Connects to the Backend

Template code	Backend value	Effect
{% if task == 'qa' %}selected{% endif %} on each option	task	The drop-down keeps the task the user just used.

Template code	Backend value	Effect
{{ prompt }} inside the textarea	prompt	The query stays in the box, so the user can edit and resubmit.
{% if response %} ... {% endif %}	response	The answer box is hidden on first visit and shown after a request.
Model: {{ model_used }}	model_used	Shows LaMini, the Gemini label, or System Error.
{{ response }}	response	Prints the answer. Jinja2 escapes HTML by default, so generated text cannot inject markup into the page.

The form uses method="post" and action="/generate", and its field names task and prompt must match the parameters in Main.py. The option values (concept, qa, summary, quiz, path) must match the TaskType enum exactly.

5. Styling (style.css)

Rule	Setting	Why it helps
body	Arial, background #f4f6f9, 40px padding, flex centering	Soft grey page with the card centred horizontally
.container	White, max-width 750px, 30px padding, 10px radius, light shadow	A readable card width that looks like an app panel
h1 and .subtitle	Dark blue-grey title, grey subtitle, centred	Clear hierarchy at the top
label	Bold, block, dark blue-grey	Labels sit above the fields and are easy to read
select, textarea	Full width, 10px padding, 6px radius, border-box	Consistent, comfortable inputs
textarea	110px height, vertical resize only	Enough room for typing without breaking the layout
button	Full width, blue #3498db, white bold text; darker #2980b9 on hover	A clear primary action with visual feedback
.response-box	Light blue background, 5px blue left border, pre-wrap, line-height 1.6	Stands out as the result; keeps the line breaks and bullet layout of the AI answer
.badge	Grey rounded label, small bold text	Shows the model name without distracting from the answer

6. User Interaction Flow

Landing	GET / shows the form with Concept selected and no answer box.
Select and type	The user chooses a task and enters a query.
Submit	A normal form POST is sent to /generate; the page reloads when the server replies.
Waiting	The first Concept request is slower because the local model loads; there is no loading indicator yet.

Result	The same page returns with the answer, the model badge, the selected task and the original query.

7. Why this Frontend Design is Useful

- **Low learning curve:** one form, two inputs and one button.
- **Transparency:** the drop-down and the badge both name the model used.
- **Continuity:** the selected task and the query are kept after submitting.
- **Reliable display of structured answers:** pre-wrap keeps quizzes, answer keys and roadmaps readable.
- **Lightweight:** no JavaScript framework, so the page loads quickly and is easy to maintain.

8. Current Limitations and Possible Improvements

- Gemini answers use Markdown symbols (headers, bullets). The page prints them as plain text, so a Markdown renderer would make results cleaner.
- The CSS has no media queries; the card adapts through width: 100% and max-width, but a dedicated mobile layout could be added.
- A loading spinner or disabled button would show that a request is in progress.
- A copy button and a history of earlier queries would improve everyday use.